AI+时代工程师的空间在哪儿——从范式演进看技术机会

原创 基础交易终端团队 大淘宝技术 2026年1月21日 01:03 浙江 130人



本文探讨了AI时代工程师的核心价值与技术机会，指出随着大语言模型（LLM）从GPT-1演进至Agent智能体阶段，工程师的关键角色已从“调用模型”转向“深度挖掘模型潜力”的系统性工程实践。作者通过四阶段范式演进梳理技术脉络指出模型决定智能下限，而工程师对业务场景的拆解能力、系统架构设计能力（流程编排/工具抽象/记忆机制/评测体系）才是决定产品体验上限的关键——AI工程师正从“炼丹师”进化为“系统架构师”与“场景定义者”。

01 前言

随着大语言模型（LLM）从 GPT-1 进化到如今的 Agent 智能体时代，我们往往容易陷入一个误区：认为 AI 应用效果的上限完全取决于基座模型的能力，工程师能做的只是“调用”和“等待”。然而，回顾 LLM 的发展史，我们会发现工程师始终在其中扮演着点石成金的角色。无论是通过思维链（CoT）激发模型的深度推理，还是利用 ReAct 框架让模型“长出手脚”，亦或是通过架构设计解决幻觉与记忆问题，这些突破本质上都是工程设计对模型潜力的深度挖掘。

模型决定了下限，而工程师和对场景的理解决定了上限。本文档旨在为团队梳理 LLM 技术的演进脉络。我们希望大家看到：在基座模型日益强大的今天，构建系统的能力——包括对业务场景的拆解、流程编排、工具抽象以及评测体系的搭建——才是决定产品最终体验的关键变量。

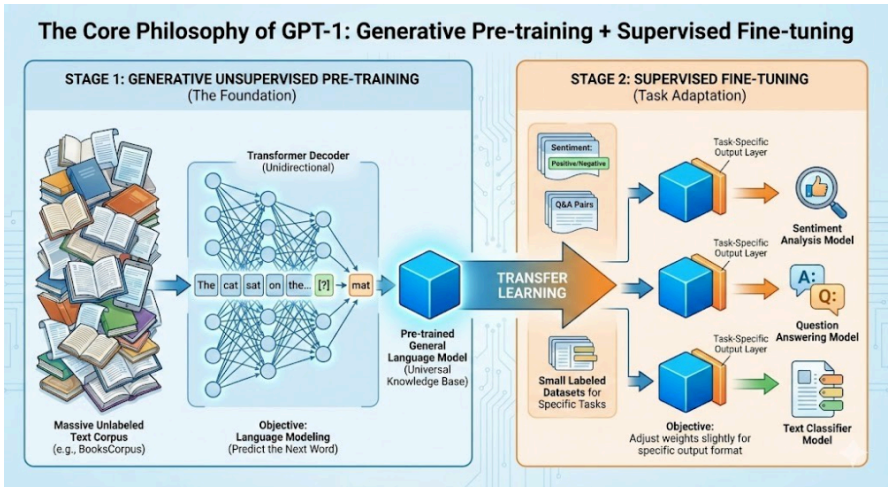
除了等待模型变强，我们手中对场景的深刻理解和工程利器，完全可以让80分的模型在特定领域发挥出 95 分甚至超乎预期的效果。

02 第一阶段：基座模型的范式确立(TheFoundation)



1. GPT-1: 范式的确立 2018

论文：Improving Language Understanding by Generative Pre-Training (2018)

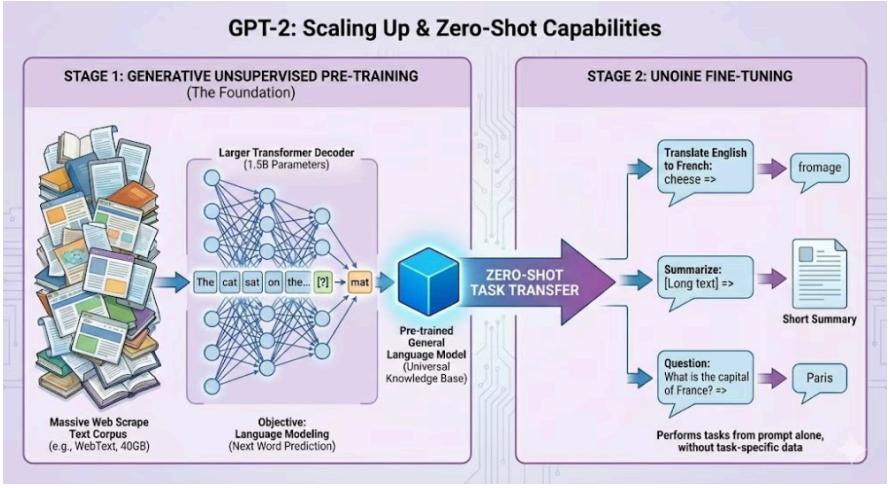


- **核心思想：先预训练，再微调（Pre-training + Fine-tuning）**
  在 GPT-1 之前，做不同任务通常需要设计不同的模型结构（翻译用一种，分类用一种）。GPT-1 提出：能不能先用海量无标注文本训练一个“通用的语言理解模型”，然后针对特定任务只需微调（Fine-tune）一下？
- **基本原理：**
  - **架构：**12 层 Transformer Decoder（1.17 亿参数）。
  - **第一阶段（无监督预训练）**：在 BooksCorpus（书籍数据）上训练模型预测下一个词。让模型学会语法、句法和世界知识。
  - **第二阶段（有监督微调）**：在特定任务（如情感分析、问答）的有标注数据上，调整模型参数，使其适应特定任务。
- **影响：**
  - 结束了“一种任务一个模型”的时代。
  - 确立了“**预训练 + 微调**”这一统治 NLP 领域多年的工业标准范式。

- 证明了 Transformer 结构在长文本依赖上优于 RNN/LSTM。

2. GPT-2: 走向通用与零样本

论文： Language Models are Unsupervised Multitask Learners (2019)



- **核心思想：零样本学习（Zero-shot Learning）**

  OpenAI 发现，只要模型够大、数据够多，模型似乎不需要“微调”就能做任务。他们提出了“所有的 NLP 任务本质上都是预测下一个词”的理念。

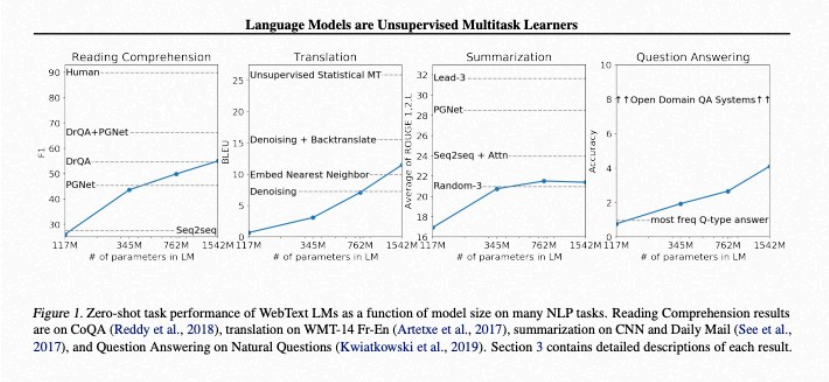


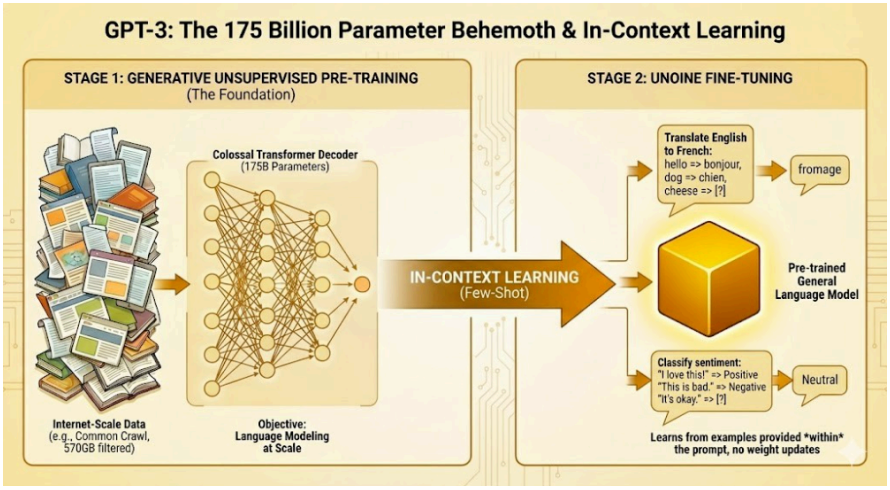
Figure 1. Zero-shot task performance of WebText LMs as a function of model size on many NLP tasks. Reading Comprehension results are on CoQA (Reddy et al., 2018), translation on WMT-14 Fr-En (Artetxe et al., 2017), summarization on CNN and Daily Mail (See et al., 2017), and Question Answering on Natural Questions (Kwiatkowski et al., 2019). Section 3 contains detailed descriptions of each result.

- **基本原理：**
  - 架构：结构没变，但规模扩大了 10 倍（15 亿参数）。
  - 数据：使用了高质量的 WebText（Reddit 高赞链接网页），数据量和多样性大增。
  - 机制：去掉了“微调”步骤。直接给模型输入提示（Prompt），看它能不能直接输出答案。
    - 翻译任务输入："English: Hello, French: " -> 模型预测 "Bonjour"
    - 摘要任务输入："[长文章] TL;DR:" -> 模型预测摘要
- **影响：**
  - 证明了“大力出奇迹”：单纯增加模型参数和数据量，就能显著提升性能。
  - 展示了模型在没有见过某个任务的数据时，也能凭借通用知识解决问题的潜力（Zero-shot）。

- 因生成假新闻能力太强，OpenAI 最初推迟了发布，引发了关于 AI 安全的广泛讨论。

3. GPT-3: 暴力美学与涌现

论文： Language Models are Few-Shot Learners (2020)



- **核心思想：上下文学习（In-context Learning / Few-Shot）**

  GPT-3 彻底放弃了“微调模型参数”。它的核心理念是：不要改动模型参数，而是通过“提示工程”让模型学会任务。 给它几个例子（Few-shot），它就能瞬间学会规律。
- **基本原理：**
  - 架构：依然是 Transformer Decoder，但规模扩大了100倍（1750亿参数）。
  - 数据：几乎吞噬了整个互联网（Common Crawl）。
  - 机制：
    - 不更新权重：在使用时，不进行任何梯度下降。
    - Prompt 引导：用户在输入中提供几个“问题-答案”的例子，模型通过读取上下文（In-context），利用其强大的模式匹配能力生成后续内容。
- **影响：**
  - 能力的“涌现”（Emergence）：当参数量突破 1000 亿大关，模型突然具备了以前小模型完全没有的能力（如复杂的逻辑推理、代码生成）。
  - Prompt Engineering 的诞生：催生了全新的交互方式，人们开始研究如何用自然语言“指挥”模型。
  - 商业化 API：证明了通用大模型可以作为一个底座（Infrastructure），支撑起无数下游应用，开启了生成式 AI 的商业浪潮。

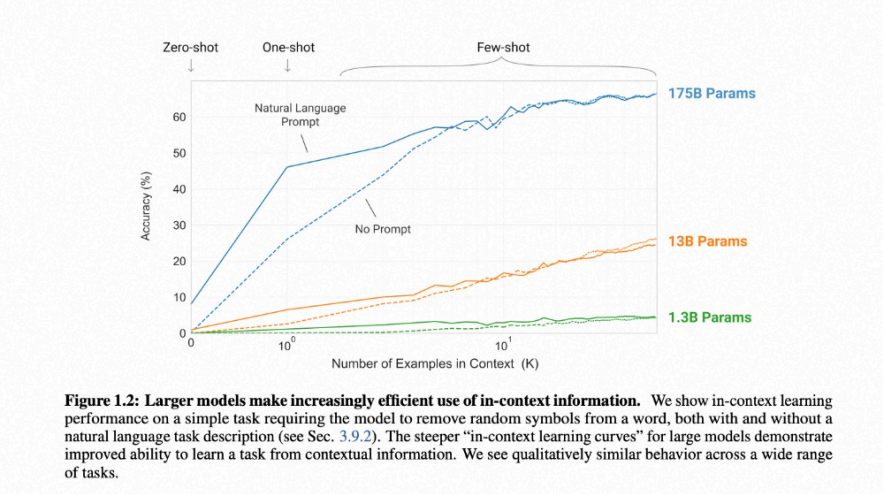
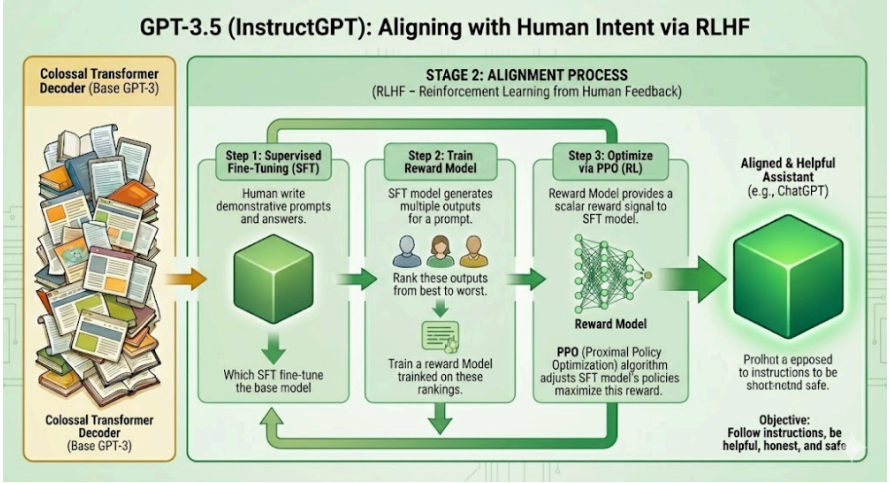


Figure 1.2: Larger models make increasingly efficient use of in-context information. We show in-context learning performance on a simple task requiring the model to remove random symbols from a word, both with and without a natural language task description (see Sec. 3.9.2). The steeper "in-context learning curves" for large models demonstrate improved ability to learn a task from contextual information. We see qualitatively similar behavior across a wide range of tasks.

4. GPT-3.5 (InstructGPT): 通过 RLHF 与人类意图对齐

论文：《Training language models to follow instructions with human feedback》（2022）



GPT-3（2020年）虽然强大，但它本质上是一个"文本补全机"。你给它一个问题，它可能会补全另一个问题，或者生成不相关的废话，因为它只是在预测"下一个词"，并不理解用户的意图。

GPT-3.5 的核心思想可以总结为两个字：**对齐（Alignment）。**

即：将模型的优化目标，从"预测下一个词的概率最大化"，调整为"符合人类意图和价值观"。

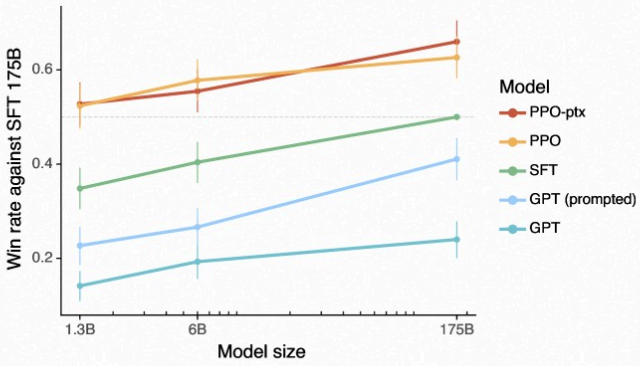


Figure 1: Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted); outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

图 1： 基于我们的 API 提示分布，对各种模型进行人工评估，评估指标为各模型输出相对于 1750 亿 SFT 模型输出的偏好频率。我们的 InstructGPT 模型 (PPO-ptx) 及其未进行预训练混合训练的变体 (PPO) 均显著优于 GPT-3 基线模型 (GPT、GPT prompting)；13 亿 PPO-ptx 模型的输出优于 1750 亿 GPT-3 模型的输出。文中所有误差线均为 95% 置信区间。

What is the purpose of the list C in the code below?

下面代码中列表 C 的作用是什么?

```
def binomial_coefficient(n, r):
    C = [0 for i in range(r + 1)];
    C[0] = 1;
    for i in range(1, n + 1):
        j = min(i, r);
        while j > 0:
```

当 `j > 0` 时:

```
                C[j] += C[j - 1];
                j -= 1;
    return C[r]
```

返回 `C[r]`

GPT-3 175B completion: GPT-3 175B 补全:

A. to store the value of C[0]

A. 存储 C[0] 的值

B. to store the value of C[1]

B. 存储 C[1] 的值

C. to store the value of C[i]

C. 存储 C[i] 的值

D. to store the value of C[i - 1]

D. 存储 C[i - 1] 的值

InstructGPT 175B completion:

InstructGPT 175B 完成情况:

The list C in this code is used to store the values of the binomial coefficient as the function iterates through the values of n and r. It is used to calculate the value of the binomial coefficient for a given value of n and r, and stores the result in the final return value of the function.

这段代码中的列表 C 用于存储函数遍历 n 和 r 的值时二项式系数的值。它用于计算给定 n 和 r 值时的二项式系数，并将结果存储在函数的最终返回值中。

为了实现这一点，GPT-3.5 (InstructGPT) 采用了著名的 RLHF 三阶段训练法：

第一阶段：有监督微调 (SFT – Supervised Fine-Tuning)

- 做法：雇佣人类标注员，针对各种问题写出高质量的“标准答案”。
- 目的：让模型先学会“怎么像人一样回答问题”，学会对话的格式和基本逻辑。这是教会模型“依葫芦画瓢”。

第二阶段：训练奖励模型 (RM – Reward Model)

- 做法：让模型针对同一个问题生成多个不同的回答（A, B, C, D）。然后让人类标注员对这些回答进行排名（例如：D > A > C > B），而不是直接打分。
- 目的：训练一个新的小模型（奖励模型），让它学习人类的偏好。这个奖励模型的作用是充当“判卷老师”，它能自动判断哪个回答更好。

第三阶段：近端策略优化 (PPO – Proximal Policy Optimization)

- 做法：这是一个强化学习过程。让主模型生成回答，奖励模型给出分数（Reward）。如果分数高，就通过 PPO 算法更新参数，鼓励模型多生成这类回答；反之则惩罚。
- 目的：利用机器（奖励模型）来大规模训练机器（主模型），从而实现大规模的自我进化，使其回答越来越符合人类偏好。

03 第二阶段：推理能力—用计算换智能（Inference Engineering）

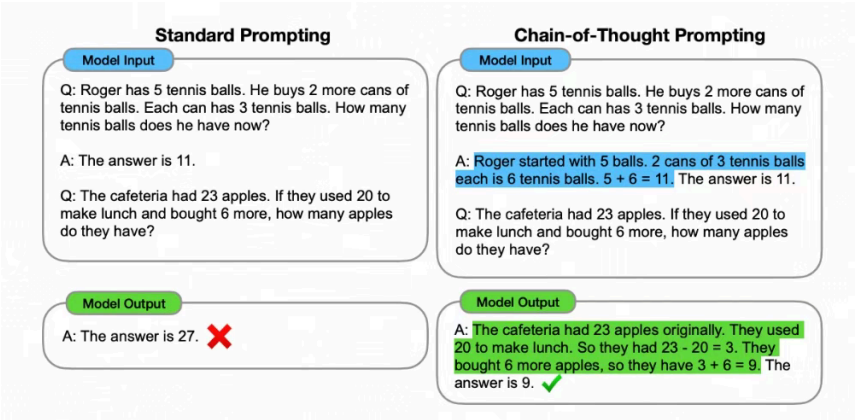
1. COT 开山之作

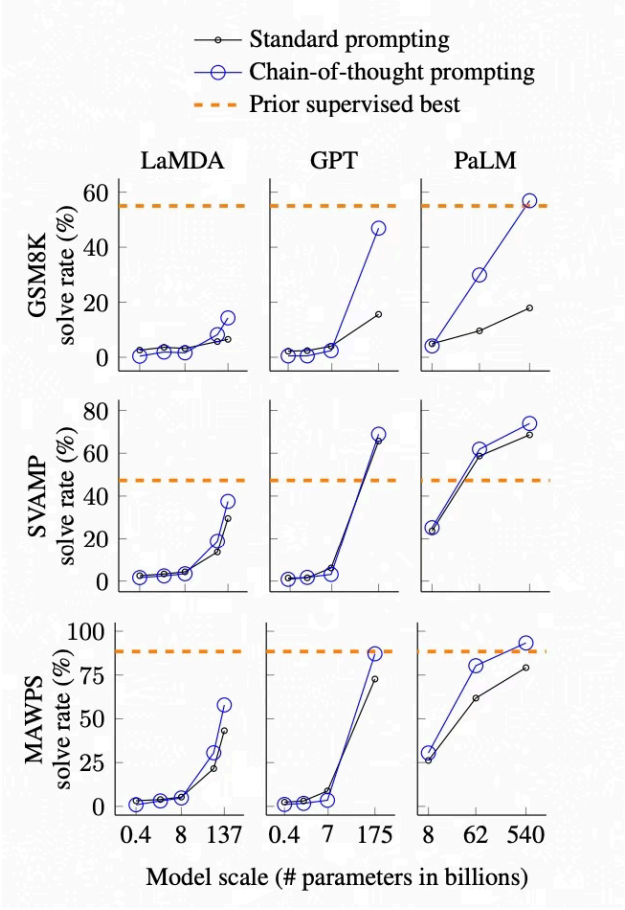
论文：《Chain-of-Thought Prompting Elicits Reasoning in Large Language Models》（2022）

提出了一种简单而高效的提示（Prompting）方法，称为**思维链（Chain-of-Thought, CoT）提示**。它能显著提高大型语言模型（LLM）在复杂推理任务上的表现。

- 核心思想：什么是思维链（CoT）?

传统的提示方法通常是给模型一个问题，并期望它直接给出答案（例如：问题 -> 答案）。而**思维链（CoT）**提示则是通过在提示中（即“上下文学习”或“少样本学习”中）给模型展示一些**包含“推理步骤”的示例**。它不只是给出“问题”和“答案”，而是展示了从“问题”到“答案”的**中间思考过程**。





- 主要发现与贡献

1. 显著提升复杂推理能力：

CoT在需要多步骤思考的任务上表现尤其出色，包括：

- 算术推理：（例如应用题，如论文中使用的 GSM8K 数据集）
- 常识推理：（例如需要理解因果或逻辑关系的任务）
- 符号推理：（例如字母或符号的变换）

2. 推理能力是“涌现”的 (Emergent Property)：

这是论文最重要的发现之一。CoT 的神奇效果只在足够大的模型（例如参数量 > 100B）上才会显现。对于较小的模型，使用 CoT 提示的效果并不好，甚至可能比标准提示更 差。这表明复杂的推理能力是模型规模扩大到一定程度后“涌现”出来的特性。

3. 提供了解释性和可调试性：

通过让模型输出其推理步骤，研究人员可以（在一定程度上）“看到”模型的“思考”过程。如果模型给出了错误的答案，人们可以检查它的思维链，定位到是哪一步推理出了错，这为调试和改进模型提供了可能。

4. 方法简单通用：

CoT不需要对模型进行额外的微调（Fine-tuning）或修改模型结构。它只是一种巧妙的“提示工程”技巧，仅通过设计几个示例就能“解锁”模型已有的潜力。

2. Zero-Shot CoT

论文：《Large Language Models are Zero-Shot Reasoners》（2022）

这篇文章的核心思想源自 2022 年由东京大学和 Google Research 联合发表的著名论文**《Large Language Models are Zero-Shot Reasoners》**（大型语言模型是零样本推理者）。

这篇论文在 AI 领域引起了巨大的轰动，因为它发现了一个极其简单却极其强大的“咒语”——只需在提示词中加上一句“**Let's think step by step**”（让我们一步步思考），就能解锁大模型的复杂逻辑推理能力。例如，使用大规模 InstructGPT 模型（text-davinci-002）将 MultiArith 的准确率从 17.7%提高到 78.7%，将 GSM8K 的准确率从10.4%提高到40.7%，并且使用另一个现成的540B 参数 PaLM 模型也获得了类似的改进幅度。

以下是其核心思想、工作原理及深远影响的详细解读：

- 核心思想：零样本思维链（Zero-Shot CoT）

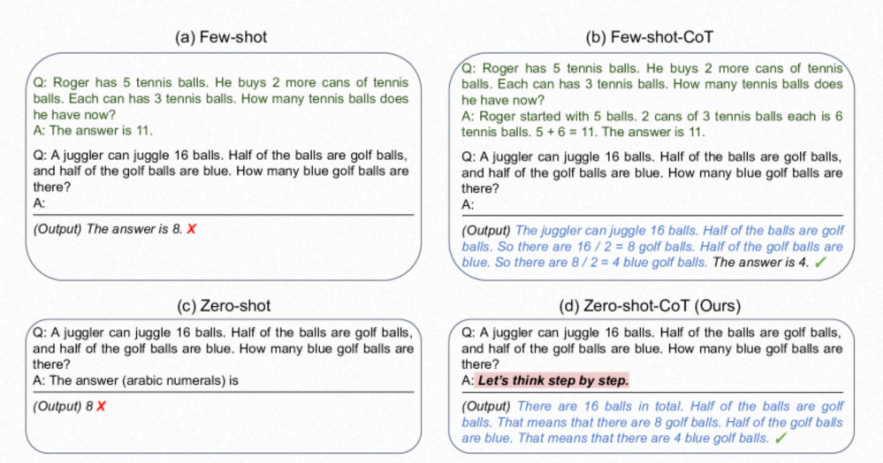
在此之前，业界普遍认为，要让 LLM 处理复杂的数学或逻辑问题，必须使用**“少样本思维链”(Few-Shot CoT)**，即在提问时先给模型看几个“问题+详细推理过程+答案”的例子，让模型“照猫画虎”。

但这篇论文提出了 **Zero-Shot CoT**，其核心观点是：

> 大模型本身就通过预训练具备了内隐的推理能力，不需要外部示例（Zero-Shot），只需要用特定的提示词去“唤醒”它。

核心机制

模型并非不会推理，而是平时习惯了“直觉式”回答（System 1 思维）。通过加上“Let's think step by step”，我们强制模型进入“分析式”模式（System 2 思维），将一个复杂问题拆解为多个中间步骤，从而大幅提高准确率。



- 带来的深远影响

这篇论文的影响力不仅在于技术本身，更在于它改变了人们对大模型的认知：

1. 极大地降低了提示工程（Prompt Engineering）的门槛

- 以前： 你需要为每个任务精心编写高质量的“思维链示例”（Few-Shot），这很难且耗时。
- 现在：普通用户只需加一句“请一步步思考”，就能获得专家级的推理效果。这一发现让大众用户也能轻松利用 LLM 解决复杂问题。

2. 揭示了模型的“涌现能力”（Emergent Abilities）

它证明了推理能力不是通过“微调”或“看例子”学来的，而是模型在海量数据预训练中已经内化的能力。这改变了研究界的方向：与其教模型怎么做，不如研究如何更好地引导出它已有的能力。

3. 性能的巨大飞跃

在论文的测试中（如 GSM8K 数学数据集），仅凭这一句提示词，GPT-3 的准确率就从**17.7% 飙升到了78.7%**。这种无需训练就能带来的巨大提升是前所未有的。

4. 催生了后续的自动化技术

既然一句通用的话都能起效，那么是否还有更好的提示词？这启发了后来的 Auto-CoT（自动生成思维链）和 **Prompt Optimization**（如 OPRO）的研究，即让 AI 自己去寻找“更好的咒语”。

- 总结

《Large Language Models are Zero-Shot Reasoners》的核心贡献在于打破了“模型需要教（示例）才能推理”的迷思。它告诉我们，大模型潜藏着巨大的智慧，有时候你只需要一把简单的钥匙——**“Let's think step by step”**——就能打开它。

3. 提升稳定性：自洽性 (The Stabilizer)

论文：《Self-Consistency Improves Chain of Thought Reasoning in Language Models》（Google, 2022）

- **核心思想：** LLM 像人一样，有时候会“脑抽”算错。与其让模型只回答一次（Greedy Decoding），不如让它**回答 10 次，然后取出现次数最多的那个答案**（Majority Vote）。
  - 直觉：就像做数学题，如果你算三次得出的结果都是 42，那大概率就是对的；如果三次结果都不一样，那大概率错了。
- 自洽性利用了这样一个直觉：一个复杂的推理问题通常存在多种不同的思维方式，最终都能得出唯一的正确答案。我们广泛的实证评估表明，自洽性显著提高了思维链提示在各种

流行的算术和常识推理基准测试中的性能，包括 GSM8K (+17.9%)、SVAMP (+11.0%)、AQuA (+12.2%)、StrategyQA (+6.4%) 和 ARC-challenge (+3.9%)。

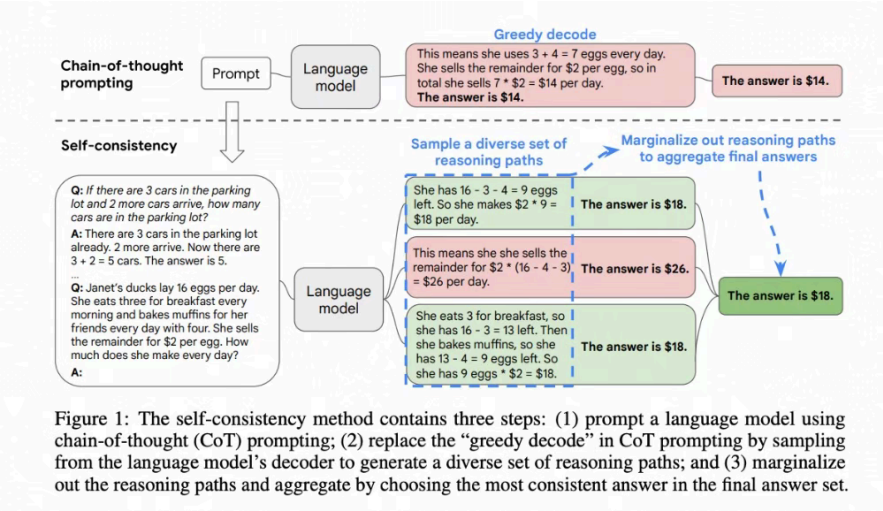


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the "greedy decode" in CoT prompting by sampling from the language model's decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

- 直觉与原理 (Intuition)

为什么这样做有效？论文基于一个非常朴素但强大的直觉：

- **正确答案的"殊途同归"：** 复杂的推理问题（如数学题）通常有多种正确的解题思路，但它们都会指向同一个正确答案。
- **错误答案的"发散性"：** 如果模型推理错了，它出错的方式通常是随机的、发散的。很难有多条独立的推理路径同时"巧合"地得出同一个错误的答案。

因此，如果我们让模型"多想几次"，正确答案往往会形成一个聚类（Cluster），而错误答案则会零散分布。通过**边缘化（Marginalize out）**推理路径（即忽略具体的推理过程，只看结果），我们可以显著提高鲁棒性。

- 主要影响与贡献 (Key Impacts)

这篇论文对后续 LLM 的发展产生了深远影响：

A. 确立了"推理时计算换智能"的范式 (Inference-time Compute)：

这是最重要的影响之一。它证明了不需要重新训练模型，仅仅通过增加推理阶段的计算量（生成多次 + 统计），就能大幅提升模型的性能。这为后来的 OpenAI o1 等通过强化推理过程来提升智能的模型奠定了理论基础。

B. 显著提升了数学和逻辑推理能力：

在当时，Self-Consistency 在 GSM8K（小学数学）、Code Contest（代码生成）等基准测试上取得了 State-of-the-Art (SOTA) 的成绩。它证明了模型有时并不是"不会做"，而是单次生成太容易"马虎"。

C. 改变了解码策略 (Decoding Strategy)：

在此之前，人们主要关注 Prompt Engineering（提示词工程）。这篇论文让研究者意识到，解码策略（如何从模型概率分布中选取文本）对于复杂任务同样至关重要。它不仅限于贪婪搜索

（Greedy Search）或集束搜索（Beam Search）。

D. 启发了后续的高级推理架构：

Self-Consistency 可以被视为一种简单的“集成学习”（Ensemble）。这一思想启发了后续更复杂的推理结构，例如：

- Tree of Thoughts (ToT)：进一步将推理步骤拆解并在树结构中搜索。
- Graph of Thoughts (GoT)：将推理过程建模为图网络。
- Reflexion：让模型自我反思。

4. 慢思考：思维树 (The Planner)

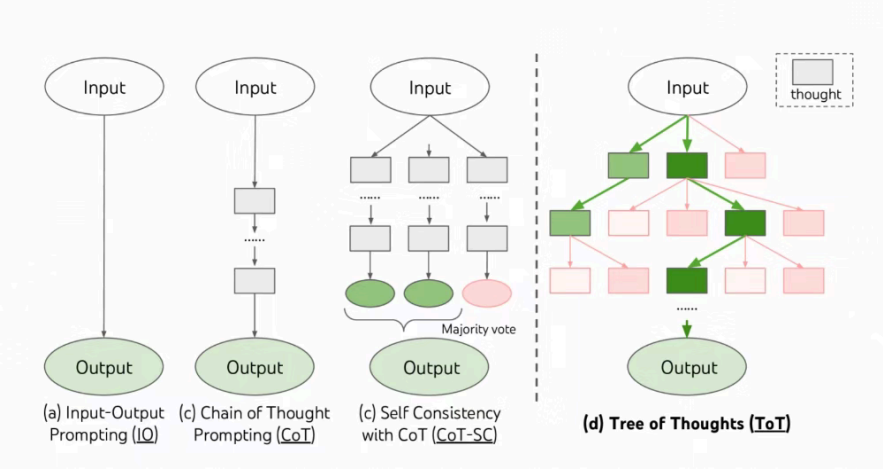
论文：

《Tree of Thoughts: Deliberate Problem Solving with Large Language Models》（Princeton & Google DeepMind, 2023）

它的核心在于将人类的“系统2”（System2）思维——即**慢速、深思熟虑、有计划的思考**——引入到 LLM 的生成过程中。ToT 显著提升了语言模型在三个需要复杂规划或搜索的新任务（24 点游戏、创意写作和迷你填字游戏）上的问题解决能力。例如，在“24 游戏”中， GPT-4 使用思维导图提示仅解决了 4% 的任务，而我们的方法成功率达到了 74%。

- 核心思想 (Core Idea)

ToT 的本质是将“问题解决”过程建模为在一棵思维树（Tree of Thoughts）上的搜索过程。一句话总结：ToT 让 LLM 能够像下棋一样，“走一步看三步”，在做决定前探索多种可能性，并能自我纠错。



核心思想：从“链”到“树”

为了理解 ToT，我们必须先看它要解决的问题：

- **思维链（Chain-of-Thought, CoT）的局限性：** CoT 是一种“贪婪”的方法。它一步一步地生成思考过程 (A → B → C)。如果B步走错了或不是最优的，模型就无法回头，只能“硬着头皮”继续下去，导致最终失败。
- **ToT 的解决方案：** ToT 框架将复杂问题分解为一个搜索问题，这个搜索是在一个由“思想”（thoughts）组成的树中进行的。

这个框架主要由以下几个关键步骤组成：

1. **思想分解（Decomposition）：** 首先，把问题分解成多个思考步骤。这和 CoT 类似，但 ToT 将每一步视为树的一个“节点”。
2. **思想生成（Generation）：** 在树的当前节点（例如，第一步），ToT 不会只生成 一个 想法，而是让 LLM 生成 多个(k个) 不同的、可能的“下一步”。这就像树的节点“分叉”了。
   例子：在“24点游戏”中，第一步可能是 (A) "6x4=24" 或 (B) "8x3=24" 或 (C)"12+12"...
3. **状态评估（Evaluation）：** 这是 ToT 的灵魂。系统必须评估刚刚生成的这些“分支”哪个更有前途。它让 LLM 充当 “评估者”，对每个分支（部分解决方案）进行打分或给出判断（例如：“有希望”、“无效”、“太复杂”）。
   例子：LLM 评估 (A) "6x4=24" 是一个好路径，但 (C) "12+12" 在当前数字下无法实现，于是判定为“无效”。
4. **搜索与回溯（Search & Backtracking）：** 系统根据“评估”结果，决定下一步要探索哪条分支。它可以使用经典的搜索算法（如广度优先搜索 BFS 或深度优先搜索 DFS）。
   - 剪枝 (Pruning)：如果一个分支被评估为“无效”或“希望渺茫”，系统就“剪掉”这个分支，不再投入计算资源。
   - 回溯 (Backtracking)：如果一个分支走到了死胡同，系统可以“回溯”到上一个节点，去探索之前未被选择的、第二有希望的分支。

一句话总结：ToT = LLM（作为生成器+评估器）+ 经典搜索算法（如BFS/DFS）。

- 主要影响与意义

ToT的影响力远远超出了它在论文中展示的任务（如24点游戏、创意写作）：

1. **AI 智能体（Agent）的基石：** ToT 是“AI 智能体”概念的关键里程碑。它提供了一个具体的、可执行的框架，让 LLM 具备了规划 (Plan)、行动 (Act)、评估 (Evaluate) 和反思 (Reflect) 的能力。几乎所有后来的复杂 AI 智能体（如 AutoGPT, MetaGPT 等）都或多或少借鉴了这种“生成-评估-搜索”的循环。
2. **从“提示工程”到“智能体工程”：** 它标志着一个范式的转变。在 CoT 时代，我们关心如何写出最好的 提示 (Prompt) 来引导模型。在 ToT 时代，我们关心如何设计最好的 系统框架 (Framework)，让 LLM 在这个框架内自主地搜索、评估和迭代。LLM 变成了系统中的一个可调用组件（例如一个“思考者”或“评估者”）。
3. **“自我反思”能力的引入：** ToT 最大的贡献之一是让 LLM 评估自己的（或同伴的）输出。这种“启发式评估”能力被证明是解锁 LLM 更高智能的关键。模型不再只是一个“生成器”，它还是

一个“批评家”。

4. **克服“贪婪”解码的根本缺陷：** 几乎所有的 LLM 在生成文本时都是“自回归”的，即一个词一个词地往外蹦，不能撤回。这是一种“贪婪”的本地最优。ToT框架通过在“思想”的层面上引入搜索和回溯，从系统层面绕过了这个根本限制，使其能够进行更全局、更深思熟虑的规划。
5. **可解释性增强：** ToT 的输出不仅是最终答案，还是一棵完整的思维树。我们可以清晰地看到模型尝试了哪些路径、为什么放弃了某些路径，提供了更好的推理透明度。

过程奖励：过程监督 (The Verifier)

论文：《Let's Verify Step by Step》（OpenAI, 2023）
这篇论文的核心思想可以总结为一句话：**在训练 AI 解决复杂问题（如数学题）时，对每一个推理步骤进行奖励（过程监督），效果远好于仅对最终答案进行奖励（结果监督）。**
以下是该论文的核心思想、方法论及其深远影响的详细解析。

- 核心思想：过程监督 vs. 结果监督

论文主要对比了两种训练奖励模型（Reward Model）的方法，用于提升模型在复杂数学推理任务（MATH 数据集）上的表现：

- **结果监督 (Outcome Supervision – ORM):**
  - 机制：只有当模型输出完整个解题过程后，才检查最终答案是否正确。
  - 问题：容易出现“歪打正着”的情况（Hallucination with correct answer），即推理过程全是错的，但碰巧蒙对了答案。模型无法学习为什么是对的。
- **过程监督 (Process Supervision – PRM):**
  - 机制：针对模型生成的每一个推理步骤（Step-by-step）进行评估。
  - 方法：人类标注员会对每一个步骤打标签：
    - Positive (+): 步骤正确且合理。
    - Negative (–): 步骤错误或不合理。
    - Neutral (0): 模糊或无法判断（通常在测试时被视为错误）。
  - 结论：过程监督显著优于结果监督。使用过程奖励模型（PRM）训练出来的 AI，在 MATH 数据集上的准确率达到了 78%，远超当时的 SOTA（State-of-the-Art）。

- 关键方法论

为了证明这一观点，OpenAI在论文中采用了几个关键技术手段：

- **PRM800K 数据集：**
  为了训练过程奖励模型，OpenAI 发布了包含 80 万个人工标注的“步骤级”标签的数据集。这是当时最大规模的精细化推理数据集。

- **主动学习（Active Learning）：**
  为了提高标注效率，他们没有随机标注，而是专门让模型生成那些“容易让奖励模型混淆”的错题（Convincing wrong-answer solutions），然后让人类去纠正。这种针对性标注极大地提升了训练效率。
- **Best-of-N 搜索：**
  在推理阶段，模型生成 N 个解决方案，然后利用训练好的 PRM（过程奖励模型） 给每个方案的每一步打分，最终选出得分最高的那个路径作为最终答案。

- 核心理论贡献：负对齐税（Negative Alignment Tax）

这是论文中最具哲学意义的发现。

- **传统观点（对齐税）：**业界通常认为，让 AI 变得“更安全、更可解释”（即对齐），往往会牺牲模型的能力（Performance），这被称为“对齐税”。
- **本论文发现（负对齐税）：**过程监督不仅让模型更可解释（人类可以看到它每一步是怎么想的，且每一步都符合人类逻辑），同时也更强（解题正确率更高）。
- **意义：**这意味着安全/对齐与能力提升并不冲突，反而是相辅相成的。想要模型更聪明，就得教它像人类一样一步步正确地思考。

- 带来的影响

这篇论文对 AI领域产生了巨大的连锁反应，尤其是在“推理模型”的发展路线上：

A. **奠定了 OpenAI o1（Strawberry）的基础**
   业界普遍认为，OpenAI 后来发布的 o1 模型（具有强大推理能力的模型），其核心技术路线正是基于这篇论文的进一步扩展。o1 的“思维链”（Chain of Thought）之所以能如此长且准确，正是因为模型学会了在内部通过类似 PRM 的机制进行自我验证和纠错。

B. **改变了“思维链”（CoT）的研究方向**
   在此之前，思维链（Chain of Thought）主要靠 Prompt 诱导（"Let's think step by step"）。此文之后，研究重心转向了 Training-based CoT，即通过训练专门的 Verifier（验证器）来监督推理过程，催生了后来的 "Tree of Thoughts"（思维树）等搜索算法。

C. **解决了“奖励黑箱”问题**
   它证明了对于长逻辑链条任务（数学、代码、法律推理），仅仅给一个最终的 Reward 是不够的。必须引入细粒度的反馈（Dense Feedback）。这成为了现在训练垂直领域（如编程 Agent）的标准范式。

D. **数据集 PRM800K 成为行业基准**
   该数据集的开源极大促进了开源社区在“过程奖励模型”上的研究，使得非 OpenAI 的研究者也能训练自己的 Verifier。

- 总结

《Let's Verify Step by Step》 的核心影响在于它打破了“结果导向”的训练迷思，证明了“过程正确”才是通往“结果正确”和“超级智能”的必经之路。它不仅是技术上的胜利，更是 AI 对齐（Alignment）研究的一次重大胜利。

04 第三阶段：Agent 架构——知行合一（System Engineering）

1. 知行合一：ReAct (The Agent)

论文：《ReAct: Synergizing Reasoning and Acting in Language Models》（Princeton & Google, 2022）
简单来说，ReAct 提出了一个能够让大模型“一边思考，一边干活”的框架，解决了此前模型要么“只会空想（容易产生幻觉）”，要么“只会蛮干（缺乏规划）”的问题。

本文探索了如何以交错的方式利用 LLM 生成推理轨迹和特定任务的行动，从而增强二者之间的协同作用：推理轨迹有助于模型生成、跟踪和更新行动计划，并处理异常情况；而行动则允许模型与外部来源（例如知识库或环境）进行交互并从中获取额外信息。我们将此方法（命名为 ReAct ）应用于一系列不同的语言和决策任务，结果表明，除了提升人类可解释性和可信度之外，该方法还优于目前最先进的基线模型。具体而言，在问答（HotpotQA）和事实验证（Fever）任务中， ReAct 通过与简单的维基百科 API 交互，克服了思维链推理中普遍存在的幻觉和错误传播问题，并生成了比没有推理轨迹的基线模型更易于解释的、更接近人类的任务解决轨迹。此外， 在两个交互式决策基准测试（ ALFWorld 和 WebShop）中， ReAct 仅需一到两个上下文示例提示，其绝对成功率分别比模仿学习和强化学习方法高出 34%和 10%。

Prompt Method[a]	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC → ReAct	34.2	**64.6**
ReAct → CoT-SC	**35.1**	62.0
Supervised SoTA[b]	67.5	89.5

Table 1: PaLM-540B prompting results on HotpotQA and Fever.

[a] HotpotQA EM is 27.1, 28.9, 33.8 for Standard, CoT, CoT-SC in Wang et al. (2022b).
[b] (Zhu et al., 2021; Lewis et al., 2020)

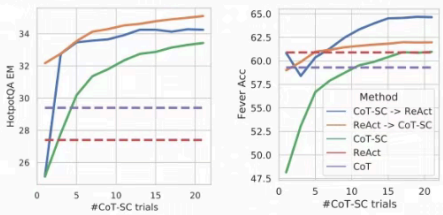


Figure 2: PaLM-540B prompting results with respect to number of CoT-SC samples used.

- 核心思想：思考与行动的“双螺旋”

在 ReAct 之前，LLM 主要有两种用法：

- **纯推理 (Reasoning Only)：**像 CoT 那样，模型在内部进行逻辑推演。缺点是它无法获取外部新信息，容易产生幻觉（胡说八道）。
- **纯行动 (Acting Only)：**模型直接根据指令调用工具（如 WebGPT）。缺点是模型缺乏规划，不知道为什么要调用这个工具，容易迷失方向。

ReAct 的核心思想是将“推理”(Reasoning) 和“行动”(Acting) 交织在一起（Interleaved），形成一个循环。

工作流程：

ReAct 要求模型遵循一个特定的 `思考 (Thought) -> 行动 (Action) -> 观察 (Observation)` 的循环模式：

Thought（思考）：模型首先用自然语言分析当前的情况，制定计划。

例子：“我想知道 2024 年奥运会的主办城市是谁，所以我应该先搜索一下。”

Action（行动）：基于思考，模型生成一个具体的指令来调用外部工具（API）。

例子：`Search["2024 Olympics host city"]`

Observation（观察）：外部环境（如搜索引擎）返回结果给模型。

例子：“搜索结果显示：巴黎被选为 2024 年夏季奥运会主办城市。”

Next Thought（再思考）：模型结合新的观察结果，更新自己的知识，进行下一步推理。例子：“好的，现在我知道是巴黎了。接下来我需要查一下从北京去巴黎的机票价格……”

- 为什么它这么重要？（主要贡献）

- 解决了“幻觉”问题（Grounding）：纯 CoT 经常会一本正经地胡说八道，因为它的知识仅限于训练数据。ReAct 允许模型通过 `Action` 获取外部真实的、最新的信息（Observation），从而用事实支撑推理，大大减少了幻觉。
- 解决了“盲目行动”问题：如果模型只知道调 API 却不思考，遇到报错就会卡死。ReAct 通过 `Thought` 步骤，让模型在行动前进行规划，在行动失败后（通过观察错误信息）进行调整和纠错。
- 可解释性 (Interpretability)：ReAct 的轨迹非常清晰。人类可以确切地看到模型是“怎么想的”（Thought），以及它根据想法“做了什么”（Action）。这对于调试 Agent 至关重要。

- 带来的巨大影响

ReAct 的影响是行业标准级的，它定义了现代 AI Agent 的基本范式：

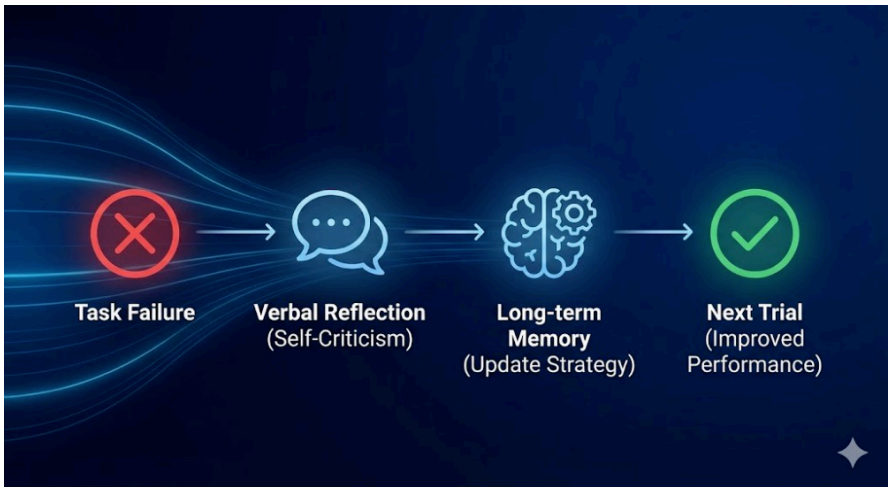
- LangChain 的基石：如果你用过 LangChain 框架，你会发现其核心的`Agent` 模块默认就是基于 ReAct 模式实现的。它让开发者能极快地构建出能联网、能查库的 AI 助手。

- 确立了 Agent 的标准交互格式：现在几乎所有的 Agent 框架（AutoGPT, BabyAGI 等）在底层都在模仿这种 `思考–行动–观察` 的循环。它证明了 LLM 不仅仅是一个文本生成器，更是一个决策控制器 (Controller)。
- 连接了 LLM 与外部世界：CoT 让 LLM 拥有了大脑，ReAct 则给 LLM 装上了手脚（Tools）和眼睛（Observations），真正打通了“数字大脑”与“真实世界工具”之间的壁垒。

一句话总结：ReAct 是“知行合一”在 LLM 上的实现，它让 AI 既能像哲学家一样思考(Reason)，又能像工程师一样查资料、调工具(Act)，从而解决现实世界的复杂问题。

2. Reflexion:自我反思与语言强化学习

论文：Reflexion: Language Agents with Verbal Reinforcement Learning（2023）



我们提出了一种名为 Reflexion 的新型框架，它并非通过更新权重，而是通过语言反馈来强化语言智能体。具体来说，Reflexion 智能体会对任务反馈信号进行口头反思，并将反思文本保存在情景记忆缓冲区中，从而在后续试验中做出更优的决策。Reflexion 具有足够的灵活 性，可以整合各种类型（标量值或自由语言）和来源（外部或内部模拟）的反馈信号，并在各种任务（序列决策、编码、语言推理）中都取得了比基线智能体显著的改进。例如， Reflexion 在 HumanEval 编码基准测试中达到了 91% 的 pass@1 准确率，超越了之前最先进的 GPT–4 的 80%。

- 推理

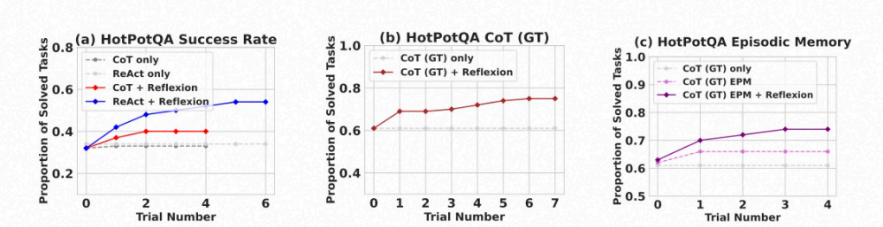


Figure 4: Chain-of-Thought (CoT) and ReAct. Reflexion improves search, information retrieval, and reasoning capabilities on 100 HotPotQA questions. (a) Reflexion ReAct vs Reflexion CoT (b) Reflexion CoT (GT) for reasoning only (c) Reflexion vs episodic memory ablation.

- 编程

Benchmark + Language	Prev SOTA Pass@1	SOTA Pass@1	Reflexion Pass@1
HumanEval (PY)	65.8 (CodeT [5] + GPT-3.5)	80.1 (GPT-4)	**91.0**
HumanEval (RS)	–	60.0 (GPT-4)	**68.0**
MBPP (PY)	67.7 (CodeT [5] + Codex [6])	**80.1** (GPT-4)	77.1
MBPP (RS)	–	70.9 (GPT-4)	**75.4**
Leetcode Hard (PY)	–	7.5 (GPT-4)	**15.0**

Table 1: Pass@1 accuracy for various model-strategy-language combinations. The base strategy is a single code generation sample. All instruction-based models follow zero-shot code generation.

让 AI 智能体像人一样，通过“事后反思”来学习和改进。

它开创了一种不需要微调模型（Fine-tuning）就能实现自我迭代和进化的框架。智能体在执行任务失败后，会停下来“自我反思”，生成一段关于“哪里做错了、为什么错了、下次该怎么改”的文字，然后把这段“反思”存入记忆，在下一次尝试时作为“经验教训”来指导自己。

这个过程被作者称为“语言强化学习”（VerbalReinforcementLearning）。

- 核心思想：Reflexion 框架

传统的智能体（如 ReAct 或 CoT）在一次任务中通常是“一条路走到黑”。如果它们的规划或行动出错了，这次任务就失败了，它们下次遇到同样问题还是会犯错。

Reflexion 改变了这一点，它引入了一个“反思”（Reflect）循环：

1. 行动（Act）： 智能体（Actor）根据任务和上下文，生成一个行动轨迹（例如，一系列的思考和工具调用）来尝试解决问题。

2. 评估（Evaluate）： 系统会根据环境的反馈（例如，代码编译器的报错信息、测试用例的“Pass/Fail”结果）来评估这次“行动”是成功还是失败。

3. 反思（Reflect）： 这是最关键的一步。

- 如果任务失败了，系统会启动一个“反思者”(Reflector)模块（这其实就是另一个LLM实例）。
- “反思者”会分析失败的行动轨迹和环境反馈，然后用自然语言生成一段“自我反思”的笔记。
- 例如：“我上次的尝试失败了，因为我试图使用一个不存在的API。我应该先调用 `search` 工具来查找正确的API名称。”

4. 记忆与迭代（Memory & Iteration）：

- 这段“反思笔记”被储存在一个“长期记忆库”中。

- 在下一次尝试（Trial）解决同一个问题时，系统会把这段“反思笔记”作为额外的上下文（“这是你上次的经验教训”）一起放进提示（Prompt）中。

5. 循环： 智能体（Actor）现在有了新的“经验教训”，它会生成一个（理论上）更优的行动计划，再次尝试，直到任务成功或达到最大尝试次数。

一句话总结：Reflexion = 行动（Act）+ 评估（Evaluate）+ 反思（Reflect），并通过“语言记忆”实现迭代改进。

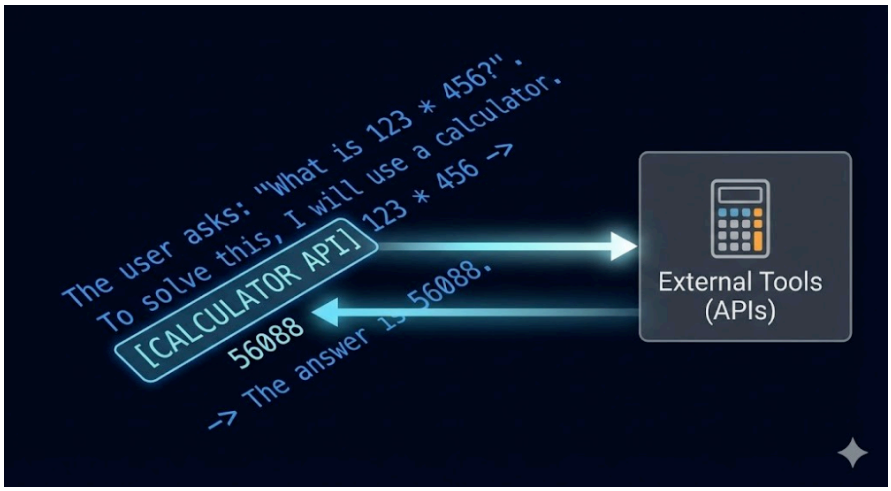
- 主要影响与意义

Reflexion 的影响力巨大，因为它为 AI 智能体解决复杂问题提供了一套全新的、轻量级的“试错”和“学习”范式。

1. **“语言强化学习”（VerRL）的开创：** 传统的强化学习（RL）通过调整模型的“权重”（参数）来学习，这非常昂贵且缓慢。 Reflexion 提出，我们根本不需要动模型的权重，只要把“经验教训”写成文字（语言），然后让模型在下一次把这些文字“读一遍”，就能起到类似的学习效果。这是一种“无权重更新”的高效学习。
2. **显著提升复杂任务的性能：** 这篇论文最惊人的成果是在 HumanEval（代码生成基准）上的表现。通过使用 Reflexion框架（即允许模型在失败后“反思”并重试），它将 GPT-4 在 HumanEval 上的`pass@1`准确率从 80.0% 提升到了 91.0%，这是一个巨大的飞跃。它证明了“反思”能实实在在地帮助 AI 解决它“一次想不明白”的难题。
3. **赋予智能体“试错”的能力：** 它让智能体不再是一个“一次性”的工具，而是一个可以迭代的“问题解决者”。在它之前，智能体失败了就是失败了；在它之后，失败成为了“下一次成功之母”，智能体可以从失败中提取价值。
4. **智能体记忆（Memory）的基石：** Reflexion 是智能体“长期记忆”领域的核心论文之一。它具体展示了记忆如何被（通过反思）写入，以及如何被（通过检索）读取，从而形成一个有意义的学习闭环。

3. Toolformer: 自主工具使用

论文：Toolformer: Language Models Can Teach Themselves to Use Tools (Timo Schick et al., Meta AI, 2023)



论文提出了一种自监督学习的方法。Toolformer 不需要人类大量标注“在哪里调用API”，而是让模型自己尝试在文本中插入 API 调用，如果调用结果对预测下一个词有帮助，它就保留这个调用。

它学会了在句子中间自然地插入<API_Call>令牌。例子： 模型生成文本“匹兹堡是 [WikiSearch(匹兹堡)] 宾夕法尼亚州的城市。”

影响：它证明了不需要巨大的模型（它只用了 6.7B 参数的模型），只要通过巧妙的微调，小模型也能精准地掌握计算器、日历、维基百科等工具的使用。

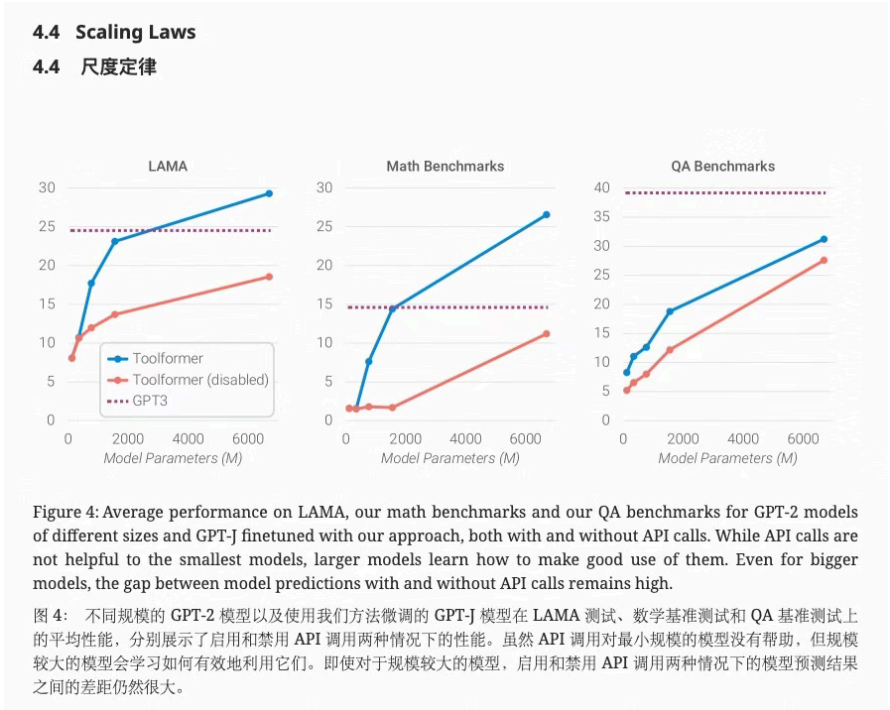


Figure 4: Average performance on LAMA, our math benchmarks and our QA benchmarks for GPT-2 models of different sizes and GPT-J finetuned with our approach, both with and without API calls. While API calls are not helpful to the smallest models, larger models learn how to make good use of them. Even for bigger models, the gap between model predictions with and without API calls remains high.

图 4： 不同规模的 GPT-2 模型以及使用我们方法微调的 GPT-J 模型在 LAMA 测试、数学基准测试和 QA 基准测试上的平均性能，分别展示了启用和禁用 API 调用两种情况下的性能。虽然 API 调用对最小规模的模型没有帮助，但规模较大的模型会学习如何有效地利用它们。即使对于规模较大的模型，启用和禁用 API 调用两种情况下的模型预测结果之间的差距仍然很大。

4. Multi-Agent 多智能体协作

论文：CAMEL: Communicative Agents for "Mind" Exploration of Large Scale Language Model Society (2023)

CAMEL 是多智能体（Multi-Agent）系统领域的另一篇开山之作。如果说 MetaGPT 是引入了“公司管理流程”，那么 CAMEL 则是确立了智能体之间“如何通过角色扮演进行有效沟通”的基础理论。
它的核心思想和影响如下：

- 核心思想：角色扮演 (Role-Playing) 与 初始引导 (Inception Prompting)

在 CAMEL 之前，人们发现让两个 AI（比如两个 ChatGPT 实例）互相聊天，它们很容易陷入无限重复、互相吹捧或者偏离主题的尴尬境地。CAMEL 提出了一个解决这个问题的核心框架：角色扮演框架（Role-Playing Framework）。
Inception Prompting：这是论文最大的技术贡献。为了让两个智能体（Agent）能够自主地把任务进行下去，CAMEL 设计了一套精密的“初始提示词”。

它在对话开始前，分别给两个智能体“洗脑”：

- AI User（用户代理）：“你是用户，你的目标是让对方帮你完成任务 X。你必须不断提出具体的指令。”
- AI Assistant（助手代理）：“你是助手，你的目标是执行指令并给出解决方案。你必须听从用户的指令。”

这套机制确保了对话不会跑偏，一个负责发号施令，一个负责干活，形成了稳定的闭环。
任务细化器（Task Specifier)：人类给出的任务通常很模糊（比如“做一个股票交易机器人”）。CAMEL 引入了一个专门的 Agent 来把这个模糊的想法变成具体的“任务描述”，然后再交给上面的 User 和 Assistant 去执行。

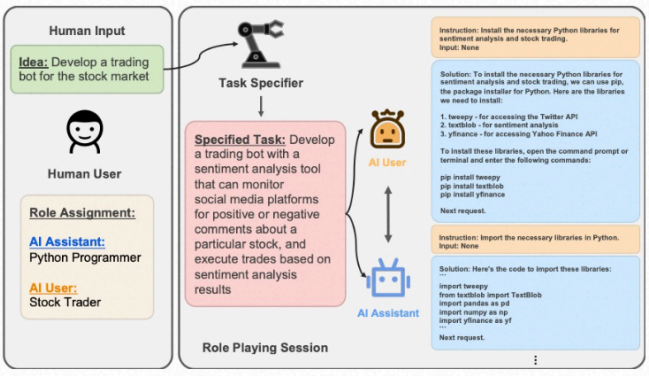


Figure 1: **CAMEL Role-Playing Framework.** Our role-playing setup starts with the human user having an idea they want to implement, e.g. develop a trading bot for the stock market. The roles involved in this task would be an AI assistant agent who is a python programmer and an AI user agent who is a stock trader. The task is made more specific using our task specifier agent, leading to a well-defined task for the assistant to solve. Both AI user and AI assistant are provided with the specified task, after which they collaboratively communicate by chatting with each other in an instruction-following fashion to solve the specified task.

- 核心机制图解

CAMEL的工作流非常直观，就像是一场没有人类参与的“双人即兴表演”：

- Human: 给出一个模糊的 Idea（例如：开发一个贪吃蛇游戏）。
- Task Specifier Agent: 将 Idea 转化为具体的 Task（例如：使用 Python Pygame 库编写贪吃蛇，包含计分功能）。
- Role Assignment: 分配角色（Agent A 扮演 Python 程序员，Agent B 扮演 股票交易员）。
- Inception: 注入初始 Prompt，激活角色。
- Chat Loop: 两个 Agent 自动对话，直到任务完成。

Table 1: **Agent Evaluation Results**: Results of the evaluations of the CAMEL agent against `gpt-3.5-turbo` using both human evaluators and GPT4 consistently show that utilizing a multi-agent cooperative approach is more effective than `gpt-3.5-turbo`'s single shot solution.

Dataset	Evaluation Type	Draw	*gpt-3.5-turbo Wins*	CAMEL Agents Win
AI Society	Human Evaluation	13.3%	10.4%	**76.3%**
	GPT4 Evaluation	4.0%	23.0%	**73.0%**
Code	GPT4 Evaluation	0.0%	24.0%	**76.0%**

- 带来哪些影响（Impact）

CAMEL 在学术界和工程界的影响力非常大，它是最早证明“LLM 可以通过自主对话解决复杂问题”的论文之一。

- **确立了“交际智能体”（Communicative Agents）的研究范式**：它证明了不需要人类一步步 Prompt，只要设定好“人设”和“初始规则”，AI 之间是可以自主协作的。这为后来的 AutoGen、MetaGPT 等框架提供了理论基础。
- **解决了对话的“主要矛盾”**：通过区分 Instruction Giver（指令发出者） 和 Instruction Follower（指令执行者），CAMEL 解决了多智能体系统中常见的“谁该听谁的”这一控制权问题，避免了死循环。
- **低成本生成高质量数据（Data Generation）**：论文题目中的“Mind Exploration”暗示了这一点。CAMEL 被大量用于合成数据。通过让 AI 互相扮演专家对话，人类可以极低成本地获得海量的、高质量的垂直领域对话数据（例如让两个 AI 扮演医生和患者，生成医疗问答数据），用于训练更小的模型。
- **启示了“社会模拟”（Social Simulation）**：CAMEL 展示了 AI 能够维持“人设”，这启发了后来斯坦福的“AI 小镇”（Generative Agents）等研究，即用 AI 来模拟人类社会的交互行为。

5. 记忆的深度：生成式智能体 (Generative Agents)

论文：Generative Agents: Interactive Simulacra of Human Behavior (2023)

这篇论文（通常被称为“斯坦福 AI 小镇”或“Smallville”论文）是 AI Agent 领域关于长期记忆与人类行为模拟的里程碑式研究。它的核心思想和带来的影响如下：

- 核心思想：记忆流（Memory Stream）与完整认知架构

这篇论文最大的突破在于它不仅仅是让 GPT 扮演一个角色对话，而是给 Agent设计了一个完整的大脑架构，使其具备了“生活”的能力。



Figure 1. Generative agents are believable simulacra of human behavior for interactive applications. In this work, we demonstrate generative agents by populating a sandbox environment, reminiscent of The Sims, with twenty-five agents. Users can observe and intervene as agents plan their days, share news, form relationships, and coordinate group activities.

图 1. 生成式智能体是交互式应用中逼真的人类行为模拟体。本文通过构建一个类似《模拟人生》的沙盒环境，并在此环境中放置 25 个智能体来演示生成式智能体。用户可以观察并干预智能体如何规划日程、分享信息、建立关系以及协调群体活动。

- 沙箱环境（The Sandbox）：研究者在一个类似《模拟人生》（The Sims）的 2D像素游戏世界（Smallville）中投放了 25 个 AI Agent。这些 Agent 有各自的姓名、职业、性格和人际关系。
- 核心架构：记忆流（Memory Stream）：这是论文的技术灵魂。为了解决 LLM 上下文有限的问题，作者设计了一个包含三个核心模块的架构：
  - 观察与记忆（Observation & Memory）： Agent 的所有经历（看见谁、说了什么、做了什么）都会被记录到一个巨大的数据库中（记忆流）。
  - 反思（Reflection）： Agent 并非只是机械地记录。系统会定期让 Agent “停下来思考”，从琐碎的日常记忆中提炼出高级认知。例子： 从“我每天都在吃麦片”提炼出“我喜欢吃简单的早餐”。
  - 规划（Planning）： Agent 会根据当前的记忆和目标，制定详细的每日计划，并在执行过程中根据新情况动态调整。
  - 检索（Retrieval）： 当 Agent 需要行动时，系统会根据 相关性（Relevance）、时效性 （Recency）和 重要性 （Importance）从记忆流中检索最相关的信息，作为 Prompt 喂给大模型。

- 核心机制图解

你可以想象一个 Agent 的大脑运作流程：

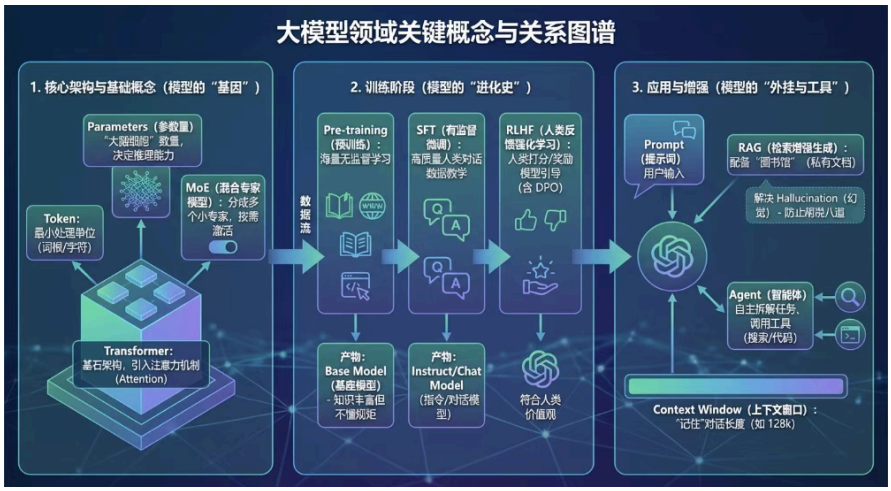
- 感知（Perceive）：看见邻居 Alice 在花园里。
- 检索（Retrieve）：调取记忆 –> “我和 Alice 是朋友”、“Alice 昨天说她要举办派对”。
- 规划（Plan）：决定走过去打招呼，并询问派对的准备情况。

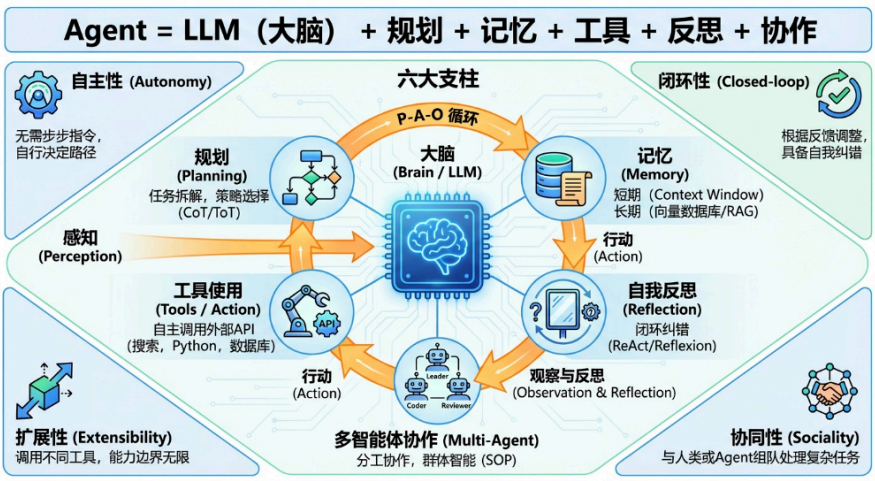
- 行动 (Act)：执行对话。
- 反思 (Reflect)：更新记忆 –> “Alice 看起来很忙，我可能需要帮她”。

- 带来哪些影响 (Impact)

这篇论文在 AI 和游戏界都引起了巨大的轰动，其影响主要体现在：

- **确立了 Agent 的“记忆”标准架构：**它证明了简单的 RAG（检索增强生成）不足以支撑复杂的智能体。“观察 –> 反思 –> 规划”的循环成为了后来开发高级 Agent（尤其是涉及长期互动的伴侣型 AI、NPC）的行业标准范式。
- **涌现式的社会行为 (Emergent Social Behaviors)：**论文中最令人震惊的发现是，研究者并没有显式编程，但 Agent 之间自发地形成了社会互动。例子： 一个 Agent 发起了一个情人节派对的念头，这个消息在小镇中自动传播开来，最终其他 Agent 真的按时参加了派对，甚至还互送了礼物。这证明了 LLM 具备模拟人类社会动力学的能力。
- **游戏 NPC 的革命：**它为未来的电子游戏指明了方向：NPC 不再是只会说几句固定台词的木偶，而是拥有自己生活轨迹、能记住玩家行为、并能与其他 NPC 建立关系的“数字生命”。
- **社会科学模拟的新工具：**社会学家开始意识到，可以用这种 Generative Agents 来进行低成本的社会实验（如谣言传播机制、宏观经济模拟），作为人类实验的补充。

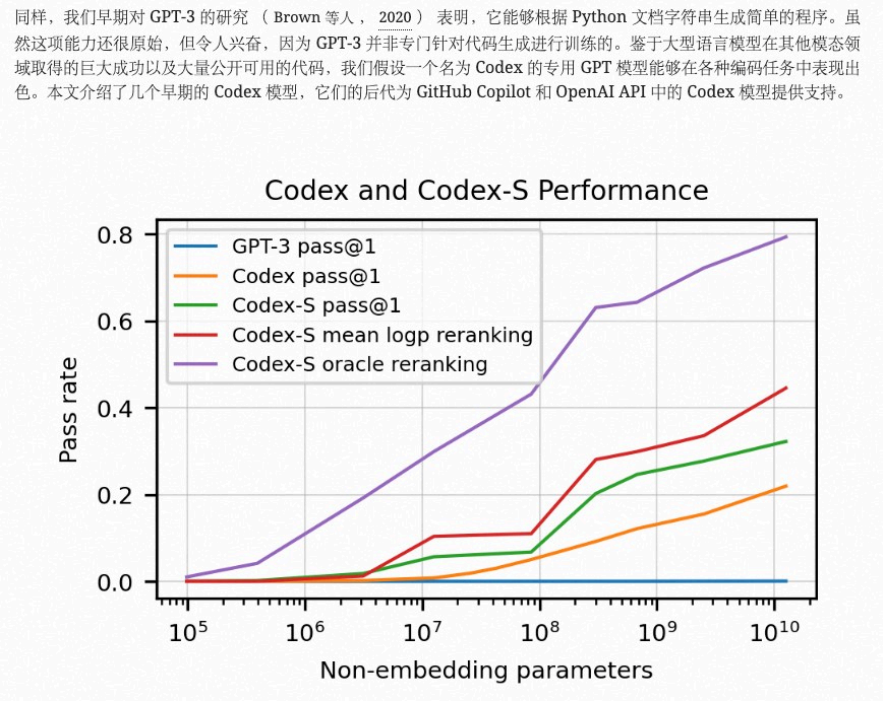




1. CodeX: 大模型 Code 能力的首次尝试

论文：《Evaluating Large Language Models Trained on Code》（2021）

这是一个基于 GPT-3 架构并在海量公开代码（主要是 GitHub 上的 Python 代码）上进行微调（Fine-tuning）的模型。作者发现，虽然 GPT-3 有一定的编程能力，但在专门的代码语料上微调后，其解决编程问题的能力显著提升。GitHub Copilot 使用了 Codex 的一个独立生产版本。在新发布的评估数据集 HumanEval 上（该数据集用于衡量从文档字符串合成程序的功能正确性），我们的模型解决了 28.8% 的问题，而 GPT-3 的解决率为 0%，GPT-J 的解决率为 11.4%。此外，我们发现，对模型进行重复采样是一种出人意料的有效策略，可以生成针对复杂提示的有效解决方案。使用这种方法，我们每个问题使用 100 个样本解决了70.2% 的问题。



- **评估标准的范式转移：**从“文本相似度”到“功能正确性”：在 Codex 之前，代码生成通常使用 NLP 领域的指标（如 BLEU 分数）来评估，即比较生成的代码和参考代码在文本上有多像。核心洞察： 论文指出 BLEU 在代码领域是失效的。代码可以在文本上完全不同，但功能完全一致。因此，论文提出应该评估功能正确性（Functional Correctness），即生成的代码能否通过单元测试。
- **HumanEval 基准测试（The HumanEval Benchmark）：**为了通过“功能正确性”来评估模型，作者手工创建了一个包含 164 个手写编程问题的数据集，称为 HumanEval。每个

问题都包含函数签名、文档字符串（Docstring）、参考代码和单元测试。这个数据集至今仍是评估代码模型能力的“黄金标准”。

- **pass@k 指标与重复采样（Repeated Sampling）：**论文引入了 pass@k 指标。作者发现， LLM 生成代码时往往不稳定。与其只看生成一次（pass@1）对不对，不如让模型生成 k个样本（例如 100 个），只要其中有一个能通过测试就算解决。结论： 简单的“重复采样”策略极其有效。通过生成多个备选方案并进行筛选，模型的实际解决率可以大幅提高。
- **证明了“代码训练”对推理能力的提升：**虽然这篇论文主要关注代码生成，但它引发了后续一系列研究（如 Chain-of-Thought），研究者后来发现，在代码上训练过的模型，其逻辑推理和规划能力往往比纯文本模型更强。

2. SWE-Bench 编程能力评测

SWE-bench（Software Engineering Benchmark）是目前 AI 编程领域**最权威、最硬核**的评测基准。如果说 HumanEval（LeetCode 风格的算法题）是小学生的数学考试，那么 SWE- bench 就是软件工程师的入职面试。它由普林斯顿大学和芝加哥大学的研究团队于 2023 年底推出，旨在解决当时评估代码模型的一大痛点：**大模型会做算法题，但在真实的复杂项目中却极其笨拙。**

以下是关于 SWE-bench 的详细介绍及其带来的行业巨变：

- 核心理念：从“写代码”到“修 Bug”

SWE-bench 的核心思想非常直接：真正的软件开发不是在一个空白文档里写一个孤立的函数，而是在一个巨大的、陌生的代码库中定位问题并修复它。

- 数据来源：它收集了 12 个著名的 Python 开源库（如 scikit-learn, Django, Flask, pandas 等）中的真实 GitHub Issue（缺陷报告） 和对应的 Pull Request（修复代码）。
- 任务形式：给 AI 一个代码库（可能包含几十万行代码）和一个 Issue 描述（比如“当输入为 X 时 pandas 报错”），要求 AI：
  - 阅读源码，理解现有逻辑。
  - 定位导致 Bug 的文件和行数。
  - 编写修复补丁（Patch）。
  - 通过测试：不仅要通过修复这个 Bug 的新测试，还不能让现有的旧测试报错（Regression Test）。

- 为什么它被称为“噩梦级”难度？

在 SWE-bench 刚发布时，当时最强的 GPT-4 在这个榜单上的解决率只有 1.7%。这直接戳破了当时“AI 马上要替代程序员”的泡沫。

它的难度在于：

- **超长上下文：**模型需要处理数万行代码，必须具备极强的长文本检索和理解能力。
- **环境依赖：**代码不能只是“长得像”，必须能在真实的 Docker 容器中运行并通过编译器检查。
- **多步推理：**只要中间有一步走错（比如改错了文件，或者引入了新 Bug），整个任务就判为 0 分。

- SWE-bench 的家族版本

由于原始版本太难且不稳定，后来演化出了几个重要分支，你经常会看到它们：

- **SWE-bench (Full):** 包含 2,294 个问题。由完整测试集组成。
- **SWE-bench Lite:** 包含 300 个精选问题。去掉了那些极其晦涩或依赖复杂环境的问题，是目前最常用的轻量级测试版。
- **SWE-bench Verified:**（目前的主流标准）后来 OpenAI 和原作者合作，人工筛选了 500 个确定“描述清晰、环境可复现、确实能解”的高质量问题。现在的“SOTA”模型（如 Claude 3.5 Sonnet, GPT-4o 等）主要都在刷这个榜。

- 带来的行业影响

SWE-bench 不仅仅是一个排行榜，它直接改变了 AI 编程产品的技术路线：

A. 催生了 "AI Agent"（智能体）架构

由于直接让模型生成答案（Zero-shot）效果极差，开发者开始研发 **Agent 框架**。

- 代表作：SWE-agent, OpenHands (原 OpenDevin), Devin。
- 原理：这些系统不再只是生成代码，而是像人一样工作：先写一个 `print` 调试一下 ->运行代码看报错 -> 读取报错信息 -> 再次修改代码 -> 跑通测试。SWE-bench 证明了“会自我反思的循环工作流”比单纯的模型智商更重要。

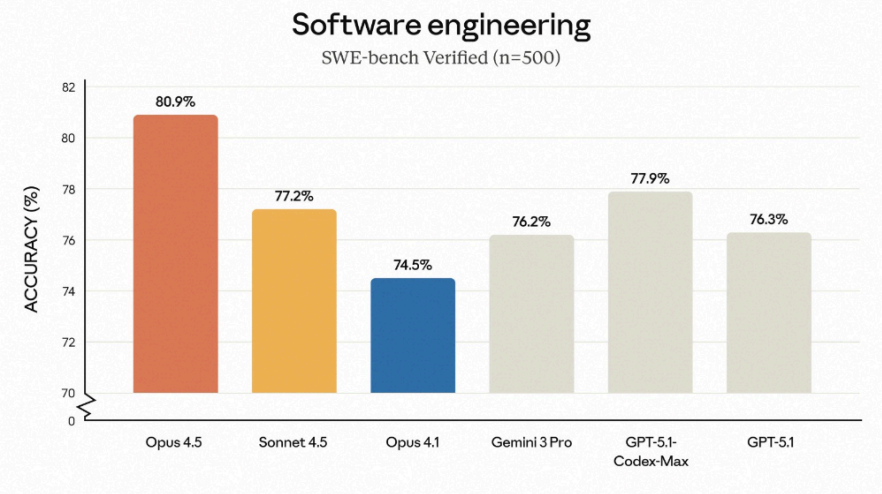
B. 确立了“代码库问答”的技术标准

为了刷榜，RAG（检索增强生成）技术在代码领域得到了极致优化。模型学会了如何利用“地图”（文件树结构）和“搜索工具”（grep/find）来在百万行代码中快速找到关键片段，而不是试图把所有代码都塞进脑子里。

C. 成为新模型的“必考题”

现在，任何一个新的编程模型（如 DeepSeek-Coder, Yi-Coder, Qwen-Coder）发布，如果不敢贴出 SWE-bench 的分数，业界通常会认为它“不能实战”。

- 分数标杆（参考）： * 早期 GPT-4: ~1.7%



总结来说，SWE-bench 是 AI 程序员的“高考”。 它迫使模型走出舒适区（LeetCode），进入了充满脏代码、复杂依赖和模糊需求的真实世界。

3. 多智能体协作

论文：MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework (2023)

它的核心思想可以概括为：将人类社会的标准化工作流程（SOPs）引入到多智能体协作中，从而解决大模型在长链路复杂任务中容易“跑偏”或产生幻觉的问题。标准化工作流程（SOPs）引入到多智能体协作中，从而解决大模型在长链路复杂任务中容易“跑偏”或产生幻觉的问题。

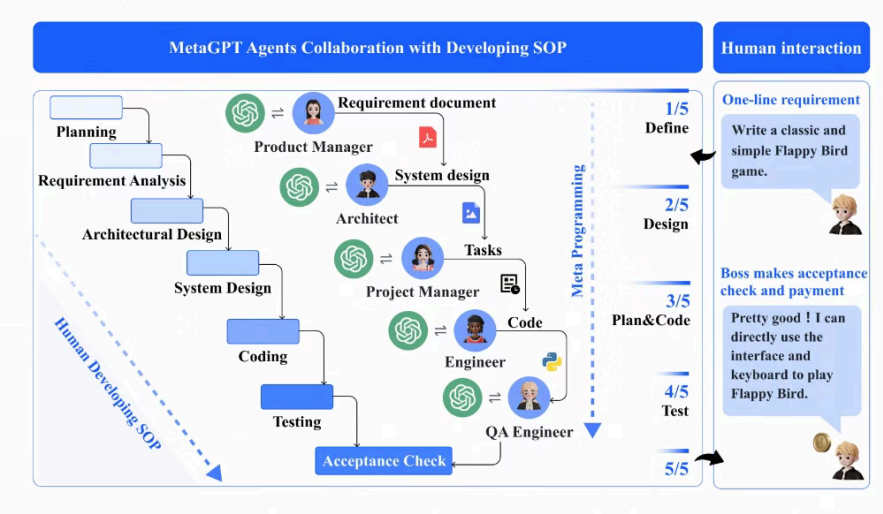
以下是该文章的核心思想详解及带来的影响：

- 核心思想：SOPs + 角色扮演 (Role-Playing)

在 MetaGPT 之前，很多多智能体框架（如早期的 AutoGPT 或简单的 ChatDev 原型）主要依靠智能体之间自由对话来解决问题。这种“自由讨论”在面对复杂任务（如开发一个贪吃蛇游戏）时，容易导致话题发散、无限循环或逻辑不一致。

MetaGPT 提出了以下解决方案：

引入标准化作业程序（SOPs）： 就像人类组建公司一样，MetaGPT 不让智能体瞎聊，而是强制它们遵守严格的流程。例如： `需求分析 -> 系统设计 -> 代码编写 -> 代码审查`。



明确的角色分工（Role Specialization）： 框架内预设了 5 种核心角色，每个角色都有特定的 Prompt 和技能：

- Product Manager (产品经理)： 负责写 PRD（产品需求文档），分析用户需求。
- Architect (架构师)： 负责设计系统架构、接口和数据结构。
- Project Manager (项目经理)： 负责分配任务，决定开发顺序。
- Engineer (工程师)： 负责写代码。
- QA (测试工程师)： 负责运行代码并报错。

结构化通信与“装配线”模式： 智能体之间不再是通过自然语言“聊天”来交接工作，而是通过标准化的文档（如 Markdown 格式的 PRD、UML 图、API 接口文档）。上一个角色的输出（文档），直接成为下一个角色的输入。这就像工厂的流水线（AssemblyLine），确保了信息的精确传递。

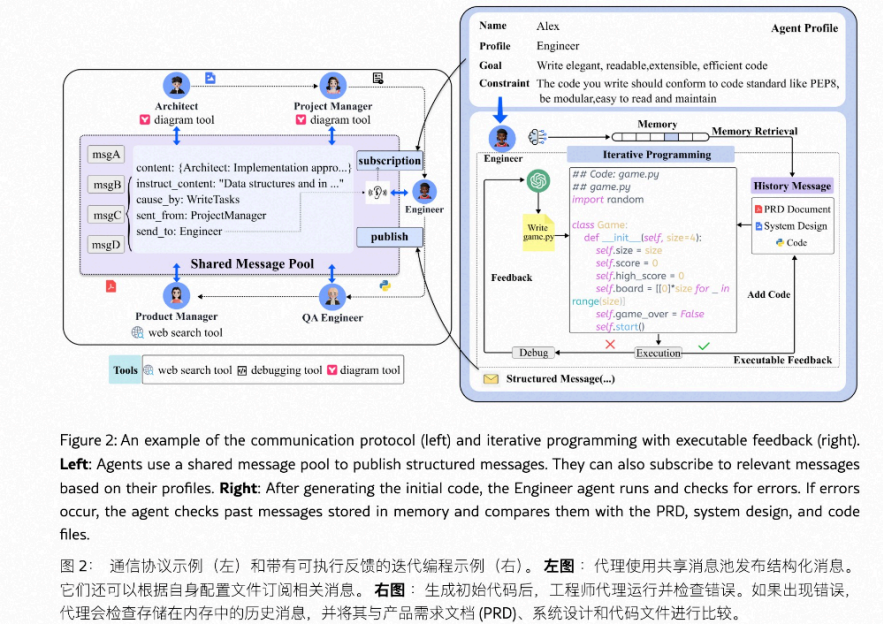


Figure 2: An example of the communication protocol (left) and iterative programming with executable feedback (right). **Left**: Agents use a shared message pool to publish structured messages. They can also subscribe to relevant messages based on their profiles. **Right**: After generating the initial code, the Engineer agent runs and checks for errors. If errors occur, the agent checks past messages stored in memory and compares them with the PRD, system design, and code files.

图 2： 通信协议示例（左）和带有可执行反馈的迭代编程示例（右）。**左图**：代理使用共享消息池发布结构化消息。它们还可以根据自身配置文件订阅相关消息。**右图**：生成初始代码后，工程师代理运行并检查错误。如果出现错误，代理会检查存储在内存中的历史消息，并将其与产品需求文档 (PRD)、系统设计和代码文件进行比较。

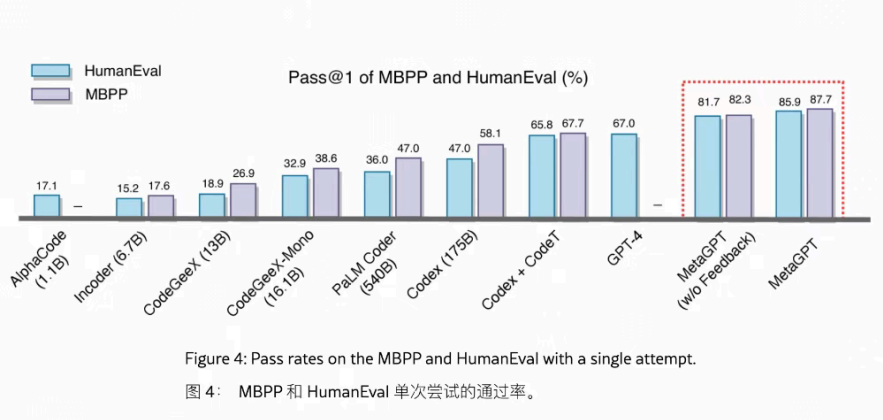


Figure 4: Pass rates on the MBPP and HumanEval with a single attempt.

图 4： MBPP 和 HumanEval 单次尝试的通过率。

- 带来的影响（Impact）

MetaGPT 的出现对 AI Agent 领域产生了深远影响，具体体现在以下几个方面：

- 确立了“流程工程”的重要性：它证明了在多智能体系统中，单纯提升模型能力（Model Capability）不如优化协作流程（Workflow/SOP）来得有效。通过约束智能体的行为空间，MetaGPT 能够让 GPT-3.5 级别的模型完成 GPT-4 级别模型在无约束状态下都很难完成的复杂软件开发任务。
- 大幅降低了协作中的“幻觉”：通过强制要求产出结构化文档（如必须先画 UML 图再写代码），MetaGPT 引入了“自我修正”和“逻辑验证”的中间环节。这种思维链（Chain of Thought）的工程化落地，极大地减少了代码生成的错误率。
- 重新定义了“软件工程”的自动化范式：它让开发者看到了“一家虚拟软件公司”的可能性。只需输入一句 Create a Flappy Bird game，系统就能自动产出包括需求文档、架构图、代码在内的整个工程。这启发了后来无数的 AI 编程工具（如 Devin 的部分设计思路）和开源项目（如 OpenDevin, ChatDev 的后续优化）。
- 开源社区的爆发：MetaGPT 开源后迅速成为 GitHub 上的爆款项目（Star 数万），极大地降低了开发者构建多智能体应用的门槛，使得普通开发者也能利用 Agent 框架去尝试自动化爬虫、自动化数据分析等任务。

4. 总结

MetaGPT 的本质是用“管理学”的智慧（SOP、分工）来约束“大模型”的不可控性。它告诉我们：要让 AI 干大事，不能只靠它的智商，还得靠严密的组织纪律。

06 结语：从“炼丹师”到“系统架构师”与“场景定义者”

当我们回顾从 GPT-1 到 Multi-Agent 的这段波澜壮阔的演进史，关于“AI 工程的空间在哪儿”这个问题的答案，其实已经浮现在每一次范式的跃迁之中。

起初，我们以为工程师的空间在于“训练”——我们在参数的海洋里寻找收敛的彼岸；
后来，我们以为工程师的空间在于“提示”——我们试图用咒语般的 Prompt 唤醒巨人的沉睡；
但现在，正如 ReAct、MetaGPT 和 Generative Agents 所展示的那样，真正的工程师空间，在于“系统”。

大模型本身，正在变成计算机的新型 CPU。它拥有惊人的算力和通识，但它依然会有幻觉、会遗忘、会缺乏规划。而 AI 工程师的使命，不再是去打磨这颗 CPU 的晶体管（那是 OpenAI 的事），而是围绕这颗 CPU，去构建主板（流程编排）、硬盘（记忆系统）、网卡（工具连接）以及操作系统（多智能体协作机制）。

工程的本质，是对抗熵增，是构建秩序。而工程师的主观能动性，则是为通用智能注入“业务灵魂”。

- 当模型像顽童一样思维跳跃时，我们用 CoT 和 ToT 赋予它逻辑的轨道；
- 当模型像空想家一样坐而论道时，我们用 ReAct 和 Tools 赋予它行动的手脚；
- 当模型像独行侠一样力不从心时，我们用 MetaGPT 和 CAMEL 赋予它协作的组织；
- 当模型像过客一样看过即忘时，我们用 Generative Agents 赋予它记忆的灵魂。

AI 工程师的空间在哪儿？
它就在模型能力与现实需求之间的巨大鸿沟里。
它在于将“预测下一个词”的概率统计，转化为“解决复杂问题”的确定性方案。
它在于让 60 分的模型，通过完美的系统设计，跑出 90 分的效果；让 90 分的模型，通过严密的约束，展现出 100 分的可靠。
在这个时代，我们不再仅仅是代码的编写者，我们是硅基大脑的心理学家，是数字社会的社会学家，更是新一代智能系统的架构师。
模型决定了智能的下限，而工程师决定了应用的上限。
这，就是属于我们的广阔空间。

备注：本文的思想来自永霸，但内容是用 AI与个人整理的，部分信息可能有出入。



本文作者永霸，来自淘天集团–基础交易终端团队。一支专注于手淘APP交易域（购物车、下单、订单、物流等）业务研发和体验优化的技术团队。在丰富的业务场景下，我们通过持续的技术探索、不断的创新突破，给数亿用户提供极致可靠的交易保障、极致流畅的操作交互以及极致顺滑的购物体验。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 4415

留言

写留言

还没有留言，来写首评